



DEVELOPER'S GUIDE: 60 TIPS OF THE SOLIDWORKS PDM API IN 45 MINUTES

60 rapid-fire tips for mastering and optimizing SOLIDWORKS
PDM add-ins.

PRESENTER



Amen Jlili

Technical Director
BLUE BYTE SYSTEMS INC.

- Mechanical Engineering background
- Developed of hundreds of add-ins, and custom integrations.
- Trainer and course author (+20h on the APIs)
- Contributor to SOLIDWORKS CSWP-API Cert. Exam
- Authored the Solid Practices Guide for PDM API



PRESENTER



Dustin Gabel

Product Manager
BLUE BYTE SYSTEMS INC.

- Mechanical Engineering background
- SOLIDWORKS PDM Administrator
- Experience supporting and leading engineering teams from small startups up to large corporations
- Manages PDM product line at BLUE BYTE SYSTEMS INC.

Link To Presentation

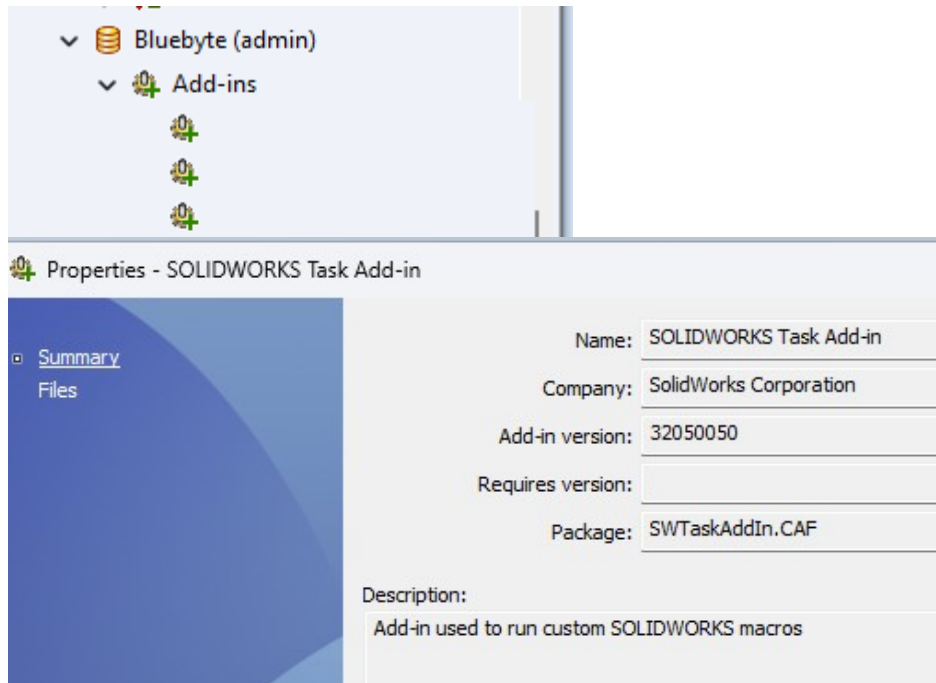
Download the presentation slides from

bluebytesystems.com/3dx



WHAT ARE ADD-INS?

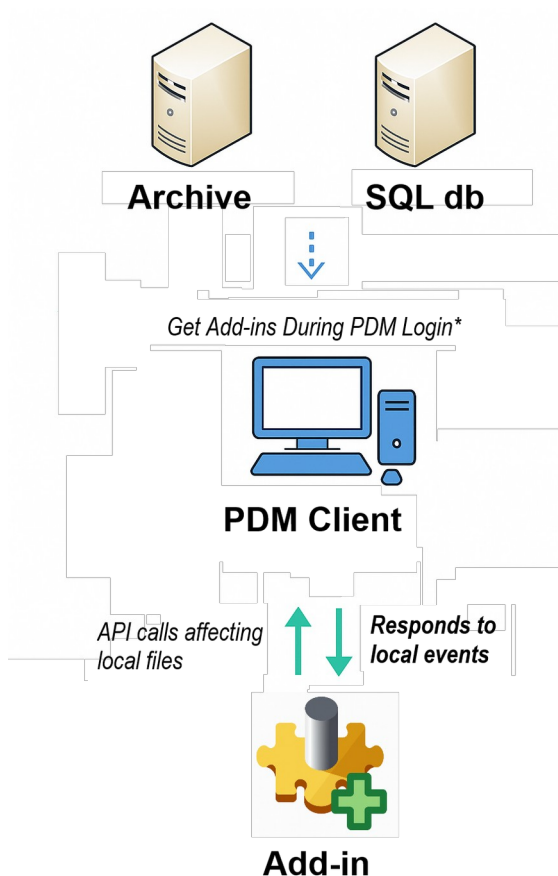
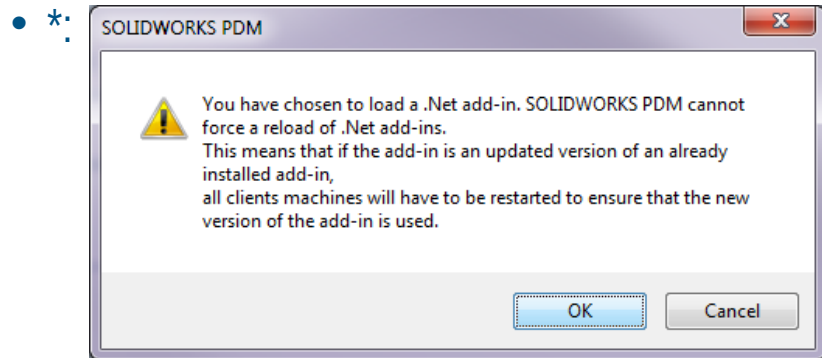
You cannot write add-ins for PDM Standard



- Compiled DLL Files (DLL: File Format)
- Loaded into the vault via Administration Tool
- Requires AdministrateAddIns permission
- Triggered by events or manual actions (Checkout, Check-in, Transitions, Menus, Datacards, etc. — ~60 events in 2025)
- Communicate with the vault using the SOLIDWORKS PDM API
- The API defines how the PDM client reacts to events.
- Built in Visual Studio.

HOW DO ADD-INS WORK?

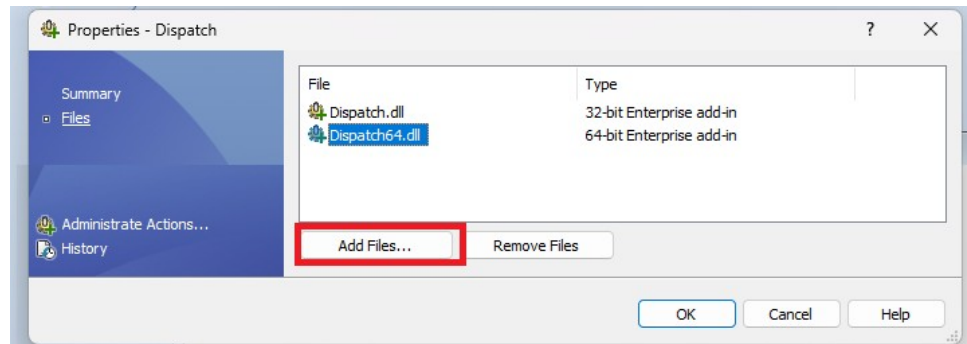
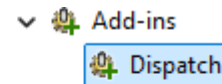
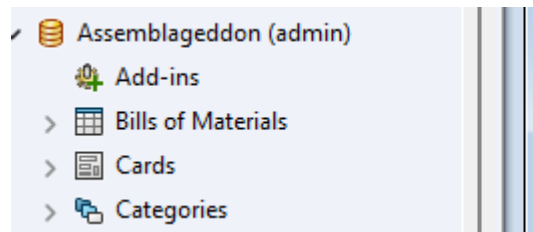
- **Add-ins are client-side applications**
- Add-ins are installed/updated from the server during the client's login
- Server cannot initiate an automatic update of add-ins loaded in a current session of PDM.



UPDATE ADD-INS

Adding add-ins is straight forward but what about updating them?

- Ideally, PDM developers are encouraged to share their updated add-ins as CEX files.
- Open in Administrator tool and drag onto the add-in node. This will replace existing version with newer.
- You can also update the DLLs by going to the add-in **Properties**, **Files** then only updating the files that have changed via **Add Files**.
- **Removing existing add-ins and re-adding them will cause the associated tasks (with the add-in) to be deleted.**



RESTARTING PDM

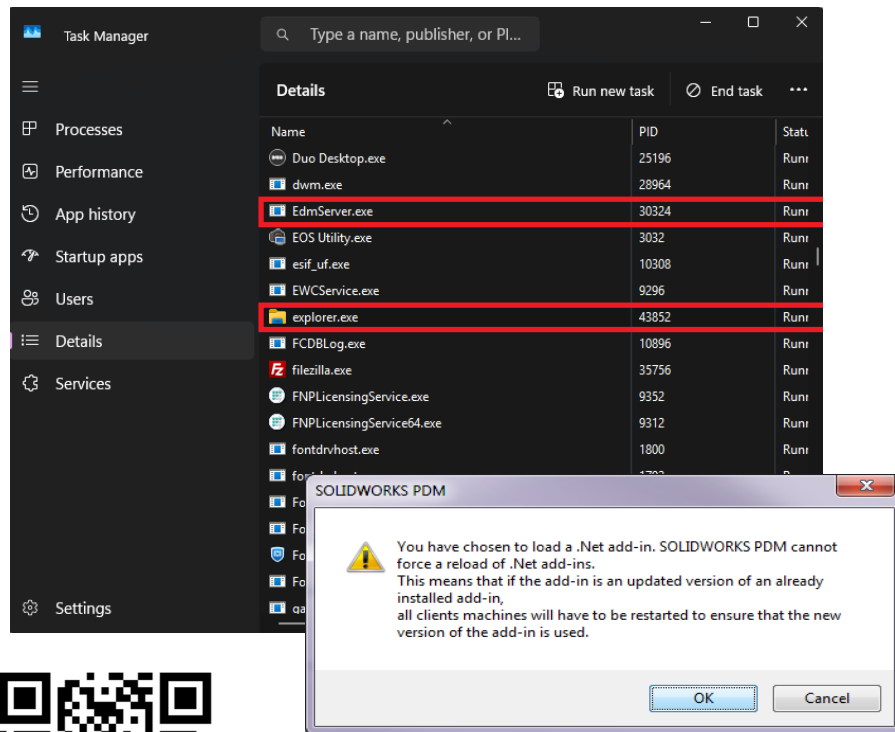
Best practice for rebooting PDM

- On local client, 80% of the time, restarting PDM is about terminating two processes:
 - **edmsvr.exe**
 - **explorer.exe**
- You must terminate all processes that consume PDM.

Script to reboot PDM:

<https://gist.github.com/AmenJlili/020178a88a8f08b22f376f63b5bae1e0>

(Thanks to Tor Iveroth 🙏)



ONCE LOADED IN MEMORY ADD-INS CANNOT BE UNLOADED

Underlying microsoft limitation

- All applications consuming PDM load add-ins (DLLs) into their AppDomain
- Loaded assemblies cannot unloaded from AppDomain : <https://learn.microsoft.com/en-us/dotnet/framework/app-domains/application-domains#unloading-an-application-domain>
- This means that the only way to **reload** an add-in to restart the host process.

Note

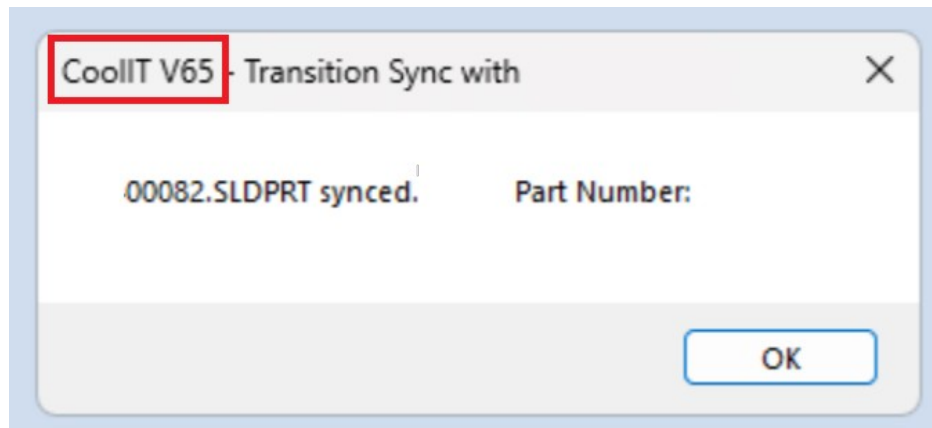
You cannot unload individual assemblies or types. Only a complete domain can be unloaded.



LOCAL ADD-IN VERSION

Make it easier for your users to report the locally installed version of your add-ins

- Include the add-in **name** and **version** in every dialog box...
- Helps users report to admin and quickly confirm which version is loaded in the current PDM session.
- Simplifies troubleshooting by making the currently loaded version of the add-in version immediately visible.



VERSIONING ADD-IN

Use yyMMdd or yyyyMMdd for your add-in version (.not the .NET assembly)

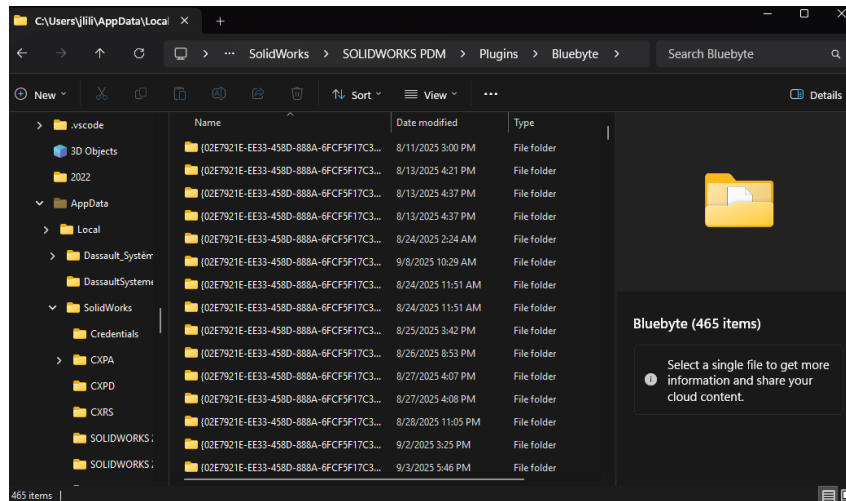
- SOLIDWORKS versions their add-ins as follow
 - CVSPBBBB
 - Where CV is the local client version (32)
 - SP is the PDM service pack (05)
 - BBBB is the build number
- You can a similar versioning schema to SOLIDWORKS if your builds are version specific.
- Using a yyMMdd might be ideal for simple add-in.

Name:	SOLIDWORKS Task Add-in
Company:	SolidWorks Corporation
Add-in version:	32050050
Requires version:	
Package:	SWTaskAddIn.CAF

CLEARING ADD-IN LOCAL CACHE

PDM add-ins are downloaded from server and saved in the client AppData folder

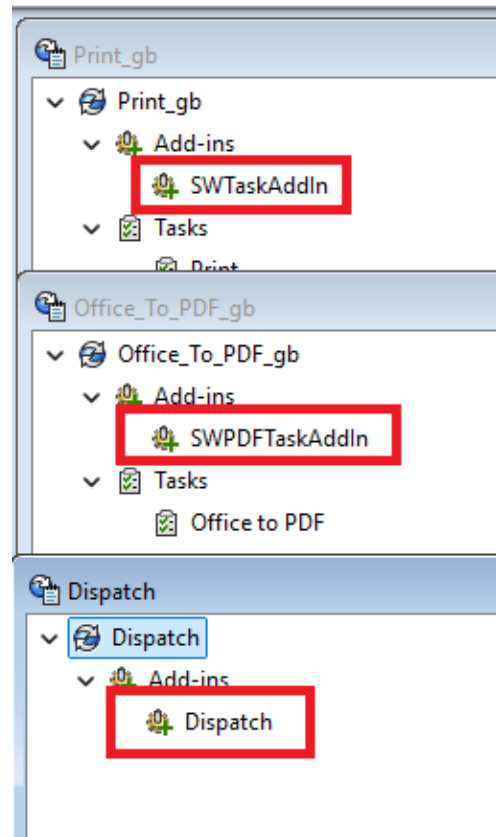
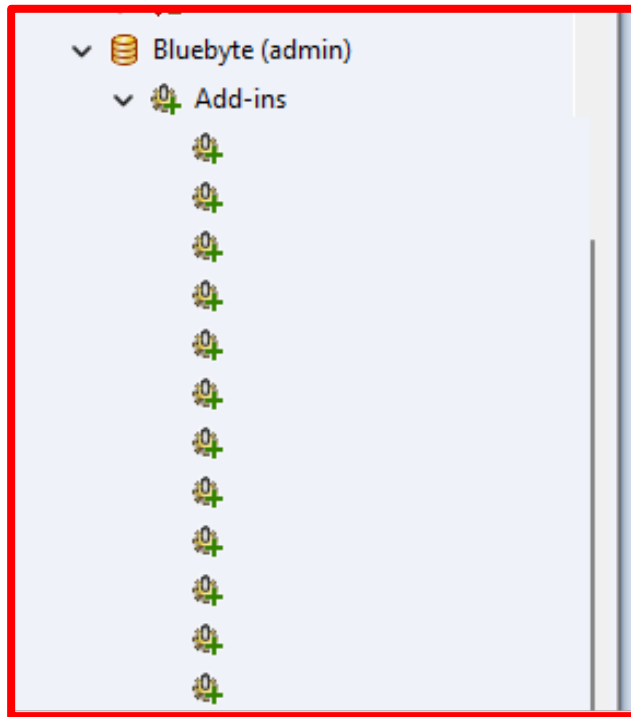
- Delete local add-in cache:
- **Close all PDM processes first**
- Open C:\Users\<User>\AppData\Local\SolidWorks\SOLIDWORKS PDM\Plugins\<VaultName>
- *AppData is hidden by default*
- Log back into the vault view to refresh the local add-in cache



IDEAL NUMBER OF ADD-INS

SOLIDWORKS PDM comes with default 3 add-ins

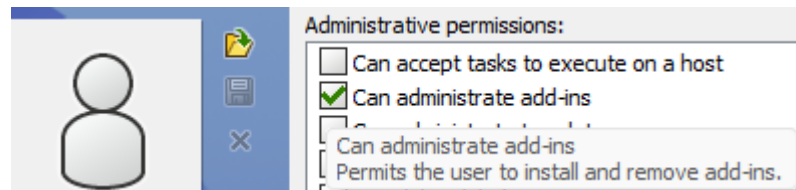
- Default add-ins are SWTaskAddIn (Convert task, design checker etc...), SWPDFTaskAddIn (Office) and Dispatch
- **There is no ideal number of add-ins from SOLIDWORKS** but what is recommended is **5** leaving PDM add-ins with **2 custom add-ins**.
- Multiple add-ins lead to longer login time especially after local add-in cache is cleared.
- Please consider consolidating custom add-ins into one add-in when possible



PREREQUISITES FOR WRITING ADD-INS

These are the system requirements and resources you need to start writing add-ins today

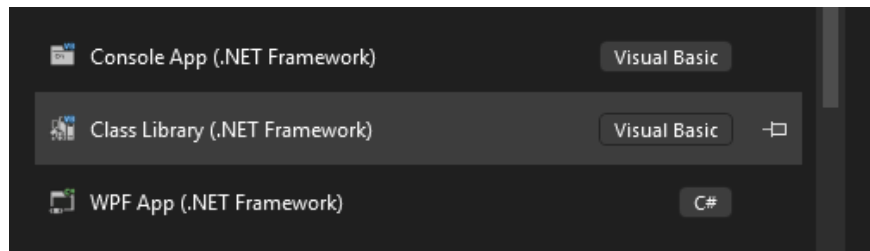
- Visual Studio Community* (22 or 26) with the desktop app component
- .NET Framework
- PDM Client;
- Administration tool;
- PDM user account that has the **AdministrateAddIns** permission
- Some knowledge of either C# or VB.NET



VISUAL STUDIO ENVIRONMENT

Choosing the correct project template

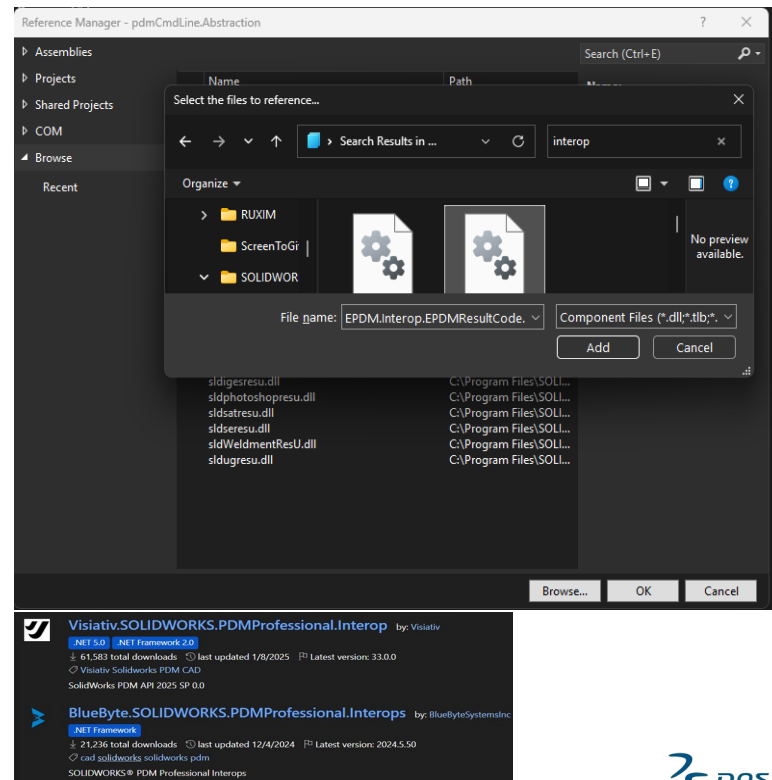
- Pick the **Class Library** project template marked with **(.NET Framework)** and your programming language
- There is no official PDM SDK provided by SOLIDWORKS.
- PDMSDK is an open source (MIT) developed and maintained by BLUE BYTE SYSTEMS INC. <https://pdmsdk.bluebyte.biz>



VISUAL STUDIO ENVIRONMENT

Use nuget packages for referencing PDM Primary Interop Assemblies (API DLL References)

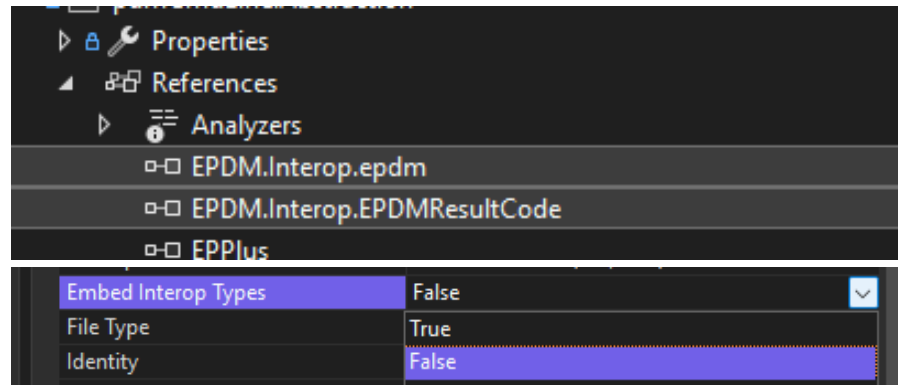
- You need two dependencies (References):
 - EPDM.Interop.epdm.dll
 - EPDM.Interop.EPDMResultCode.dll
- References found in PDM installation folder
- **Adding references from local client creates a file system DEPENDENCY on the local machine**
- **Use PDM PIAs NuGet packages (RECOMMENDATION).**
- **Visual Studio restore nuget package on build.**



VISUAL STUDIO ENVIRONMENT

Embed Interop Types = False

- **Embed Interop Types** is a Visual Studio feature that embeds COM interop types directly into your add-in's main DLL.
- Set **Embed Interop Types = False** for all PDM references.

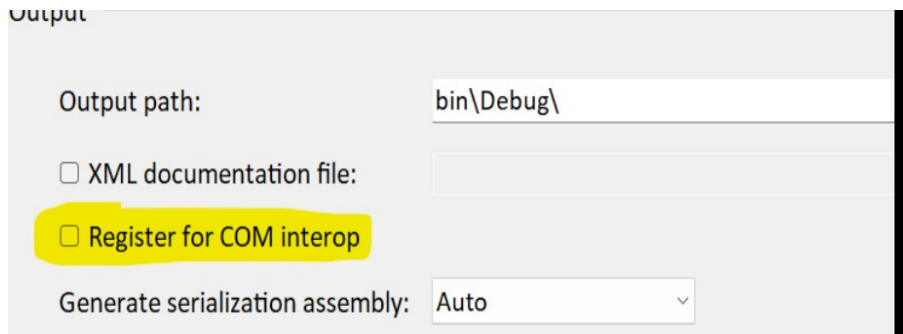


**THIS IS MOST OF THE
MOST IMPORTANT TIPS
TO REMEMBER**

VISUAL STUDIO ENVIRONMENT

Register For COM Interop

- **Register for COM Interop** (Application tab) registers your .NET DLL locally so the PDM client can use your add-in.
- **SOLIDWORKS add-ins:** Required if registering manually with Regasm.exe.
- **PDM add-ins:** not required. The Administration Tool handles registration.



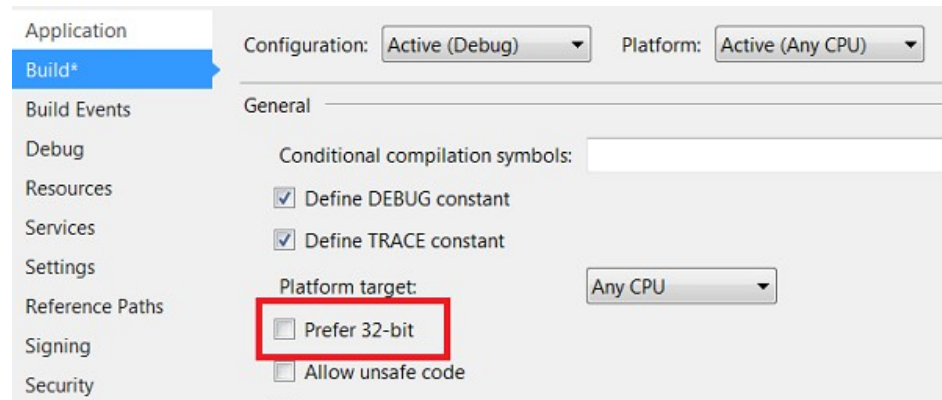
VISUAL STUDIO ENVIRONMENT

Prefer 32 bits = False (*For desktop applications*)

THIS IS MOST OF THE MOST IMPORTANT TIPS TO REMEMBER

For desktop applications :

- Console applications;
 - Winforms applications;
 - Windows Presentation Foundation (WPF) applications;
- designed with the .NET framework, make sure *Prefer 32 bits* is unchecked in the Application tab.



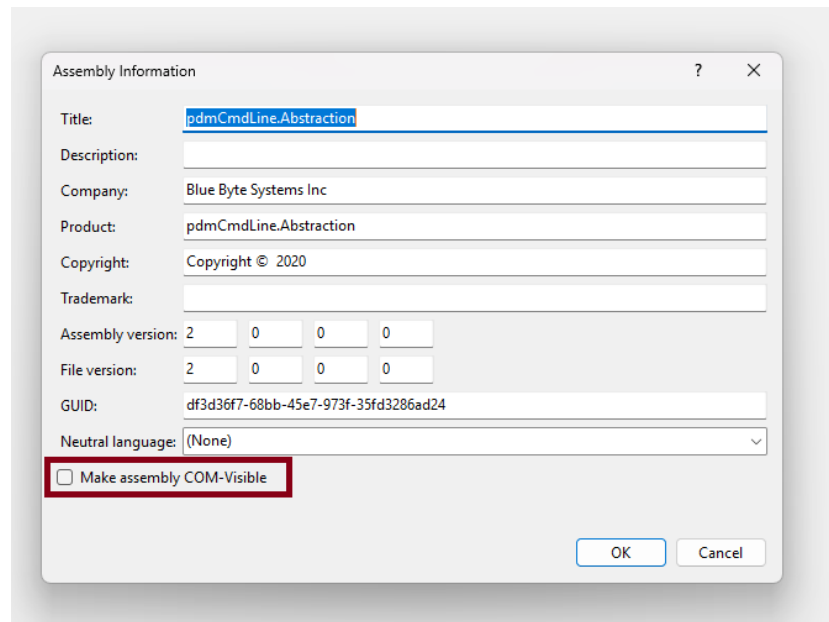
If *Prefer 32 bits* is **checked**, you may get invalid file format error:

E_EDM_INVALID_FILE 0x80040229 = The file format is not recognized

VISUAL STUDIO ENVIRONMENT

Make assembly COM-Visible

- Under the Application tab in the project properties, you can the assembly information
- This is where you the version of the DLL (**Not the PDM add-in version**) as well as other identifying information
- Please **do not** check Make assembly COM-Visible
- This will *unnecessarily* register all public classes in the local client registry.



PROGRAMMING BEST PRACTICES

Requirement for the add-in class implementing IEdmAddIn5

- The IEdmAddIn5 interface is the **contract** between your add-in and PDM.
- At least one class must implement the IEdmAddIn5
- Class must be **Public**
- Class must be decorated with a GUID
 - *To create a GUID, go Tools > Create GUID*
- Highly recommend to decorate the add-in class with the **ComVisibleAttribute** (As opposed to doing that via the Assembly Information Dialog)

```
[Guid("7DD7CC8A-AD2F-4825-9EF8-C09F784DC625")]  
[ComVisible(true)]  
0 references  
public class Main : IEdmAddIn5  
{
```

PROGRAMMING BEST PRACTICES

Securing the implementation of the IEdmAddIn5 against unhandled exceptions

```
public class Main : IEdmAddIn5
{
    public void GetAddInInfo(ref EdmAddInInfo poInfo,
        IEdmVault5 poVault, IEdmCmdMgr5 poCmdMgr)
    {}

    public void OnCmd(ref EdmCmd poCmd, ref EdmCmdData[] ppoData)
    {}
}
```

PROGRAMMING BEST PRACTICES

Securing the implementation of the IEdmAddIn5 against unhandled exceptions

```
public void GetAddInInfo(ref EdmAddInInfo poInfo,  
IEdmVault5 poVault, IEdmCmdMgr5 poCmdMgr)  
{  
    try  
    { // TODO: Command handling logic }  
    catch (Exception error)  
    { // TODO: Handle exception }  
}
```

PROGRAMMING BEST PRACTICES

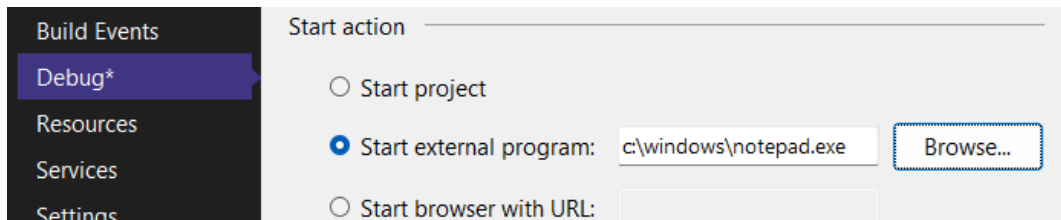
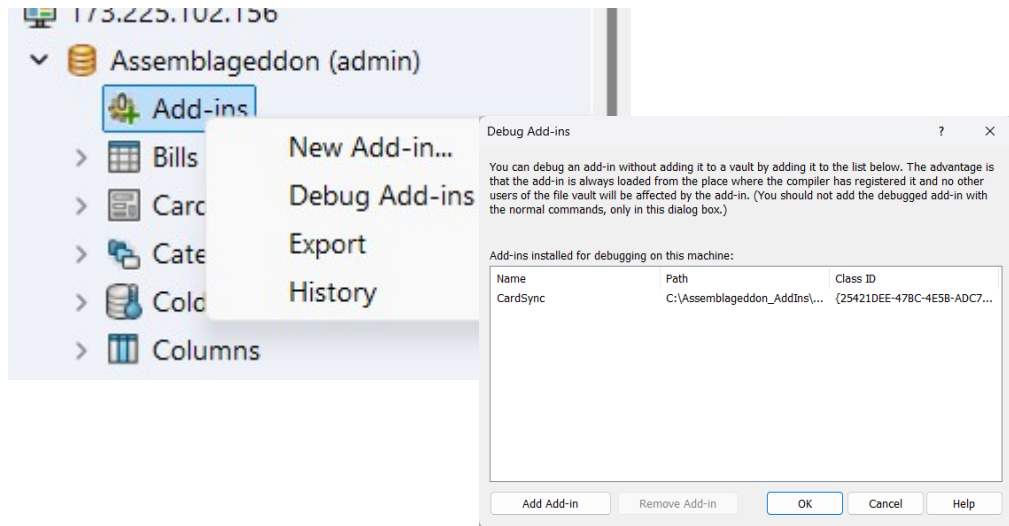
Handling exceptions

```
public void OnCmd(ref EdmCmd poCmd, ref  
EdmCmdData[] ppoData)  
{  
    try  
    { // TODO: Command handling logic }  
    catch (COMException pdmError)  
    { // TODO: handle this PDM exception }  
    catch (Exception genericError)  
    { // TODO: handle this generic exception }  
}
```


DEBUGGING

Standard debugging sequence (notepad.exe)

1. Add add-in DLL from the Administration Tool (Add-ins > Debug Add-in...)
2. Add Program notepad.exe under Debug tab in Project properties
3. Press Start
4. Notepad.exe is open, click File > Open
5. Browse to Vault and trigger add-in



DEBUGGING

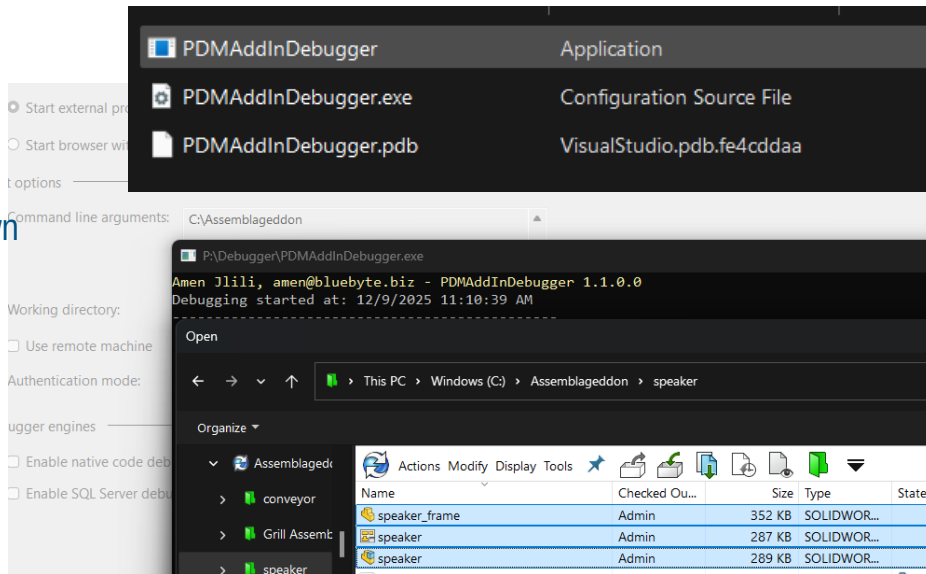
Enhanced debugging with PDMAAddInDebugger.exe (Free debugging Tool)

Notepad.exe limitations:

- Open file dialog: No automatic browsing to vault folder
- Open file dialog: No multi-select
- Open file dialog: No SOLIDWORKS extensions dropdown

PDMAAddInDebugger.exe Advantages

- You can use Console.WriteLine to the console.
- Multi-selection is enabled.
- Prefer 32 bits is unchecked



COMMON ERRORS AND EXCEPTIONS

Class not registered

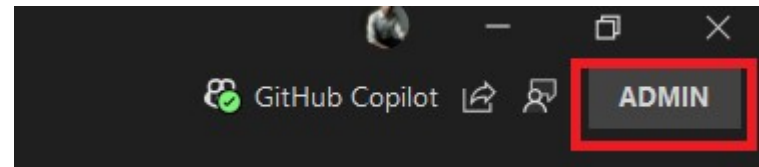
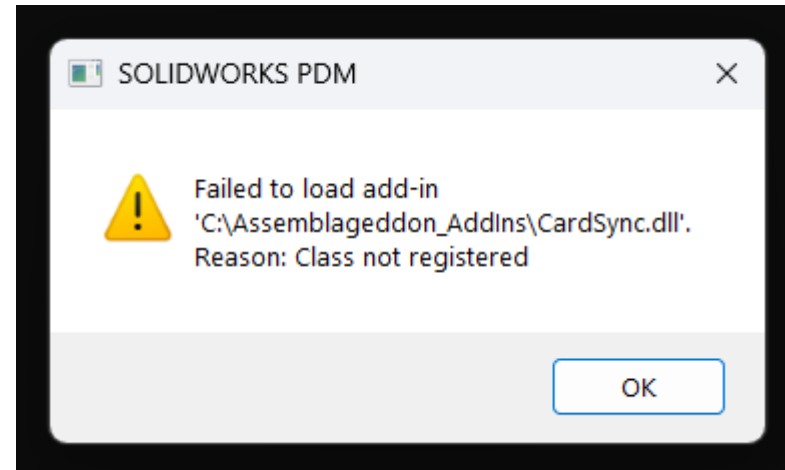
PDM client can't load add-in if it's registered under a different UAC level. Elevated registration

(HKLM: *HKLM (HKEY_LOCAL_MACHINE)* is a primary "hive" or section in the Microsoft Windows Registry database) is invisible to non-elevated PDM client.

Recommended:

Don't run Visual Studio as Administrator.

Match the UAC level of the PDM client when registering add-ins.



COMMON ERRORS AND EXCEPTIONS

The DLL is not a COM module

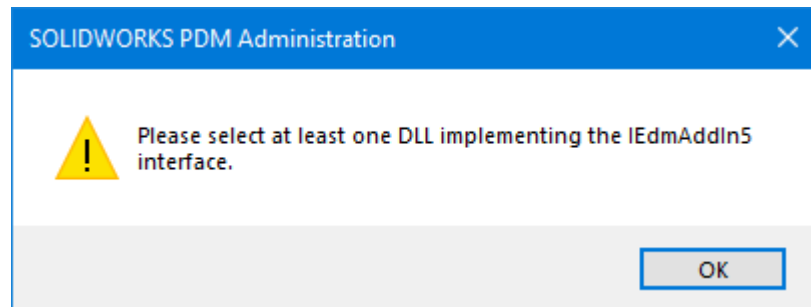
This error is quite straightforward. The class that implements the IEdmAddIn5 interface must be COM visible. The best way to expose a .NET class as COM visible is as follows:

```
{  
    [Guid("7DD7CC8A-AD2F-4825-9EF8-C09F784DC625")]  
    [ComVisible(true)]  
    0 references  
    public class Main : IEdmAddIn5  
    {
```

COMMON ERRORS AND EXCEPTIONS

Please select at least one DLL implementing the IEdmAddIn5 Interface

Every add-in must have a class that implements the IEdmAddIn5 interface. When your vault fails to find a class that implements IEdmAddIn5, this error message is displayed to the user.

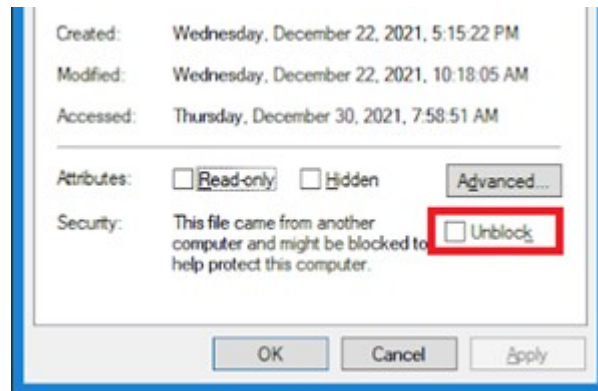
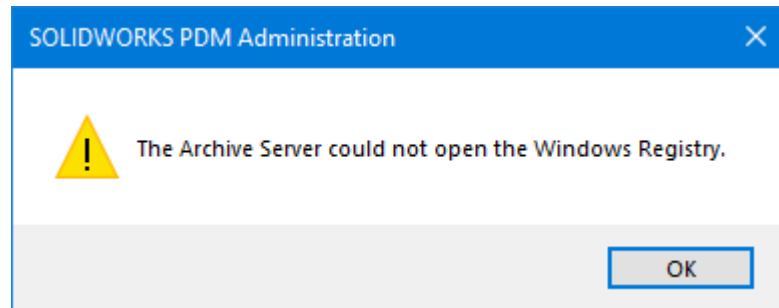


```
{  
    [Guid("7DD7CC8A-AD2F-4825-9EF8-C09F784DC625")]  
    [ComVisible(true)]  
    0 references  
    public class Main : IEdmAddIn5  
    {
```

COMMON ERRORS AND EXCEPTIONS

The archive server could not open the Windows Registry

- DLLs downloaded from another computer may be **blocked by Windows**.
- Error: *“The archive server could not open the Windows Registry.”*
- **Fix:**
- Right-click the DLL → **Properties** → **General** tab.
- If you see “This file came from another computer...”, click **Unblock**.
- Click **OK** and repeat for other DLLs.



SOLIDWORKS PDM API TIPS

What is the difference between the IEdmVault5 login methods?

- **LoginAuto**

Uses the current PDM session. No credentials needed. Ideal for user-attended apps and add-ins.

- **Login**

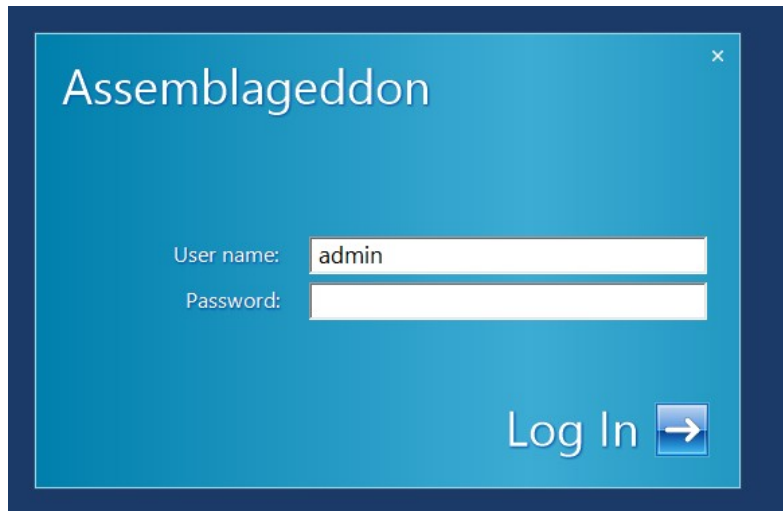
Requires **username** and **password**. Doesn't consume a license.

You must call LoginEx or LoginAuto prior to this method to consume a license.

- **LoginEx**

Requires **username** and **password**. Consumes a license.

Best for unattended or external applications.

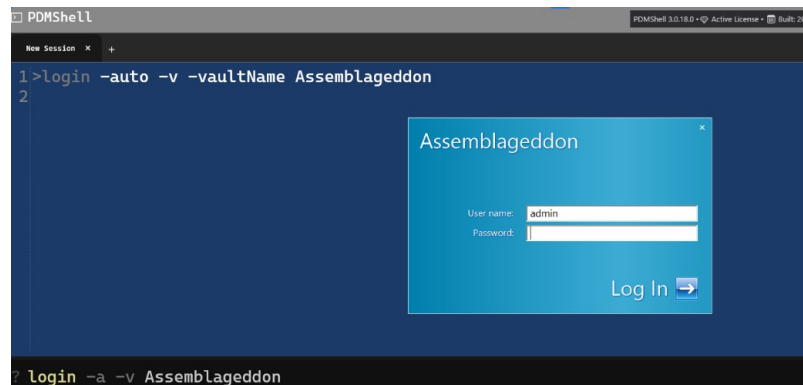


SOLIDWORKS PDM API TIPS

What is **HParentWnd**?

- **hParentWnd** appears in many SOLIDWORKS PDM API calls that display a window or error dialog.
- It represents the **handle (hWnd)** of the host window where the add-in is loaded (a Win32 window identifier).
- In File Explorer, each session gets a new handle. It is not persistent.
- In the PDM API, it anchors modal dialogs (e.g., login or check-in) to the host window.

```
Sub LoginAuto( _  
    ByVal bsVaultName As System.String, _  
    ByVal hParentWnd As System.Integer _  
)
```



SOLIDWORKS PDM API TIPS

File ID and Folder ID go together

SOLIDWORKS PDM stores files in folders.

Many API calls like GetLocalPath, and check-out (LockFile) require folder ID.

Always store both IDs when working with files.

The EdmCmdData structure usually provides both during file events.

```
Sub LockFile( _  
    ByVal LParentFolderID As System.Integer, _  
    ByVal LParentWnd As System.Integer, _  
    Optional ByVal LEdmLockFlags As System.Integer _  
)
```

Visual Basic (Declaration)

```
Function GetLocalPath( _  
    ByVal LParentFolderID As System.Integer _  
) As System.String
```

SOLIDWORKS PDM API TIPS

Always use new interfaces

The API is built with backward compatibility in mind. All interfaces start with Edm (Electronic Data Management) and end in a number.

Newer interfaces implement the obsolete version but not the other way.

IEdmVault17 will have all the methods from **IEdmVault5, 6**.

Using the latest version of the interface will avoid doing unnecessary casts between the obsolete version and the updated version throughout your code.

```
var vault17 = this.Vault as IEdmVault17;  
vault17.BrowseForFolder2(handle, "Select Folder  
if (poCm  
    BrowseForFile  
    BrowseForFolder  
    BrowseForFolder2  
IEdmFolder5 IEdmVault17.B
```

SOLIDWORKS PDM API TIPS

Searching is faster than iterating through folders (for large sets)

When reading files in a folder, you have two options:

- **Traverse the content of the folder :**
 - Iterate through `GetNextFile` until end of the list
- **Perform a search for all *.sld* files.**
 - Create Search object (`IEdmSearch5`)
 - Iterate through results with `GetNext`

```
public static void CompareTraversalMethods()
{
    // Resolve the folder
    IEdmFolder5 targetFolder = _vault.GetFolder5("C:\\Assembledaddon");
    if (targetFolder == null)
    {
        Console.WriteLine("Folder not found");
        return;
    }

    Console.WriteLine("Comparing folder traversal methods in: C:\\Assembledaddon");
    TraverseWithIterator(targetFolder);
    TraverseWithSearch(folderPath);
}

// Method 1: Iterator (GetFirstFile)
// Method 2: Search (IEdmSearch5)

Console.WriteLine("Comparing folder traversal methods in: C:\\Users\\jlili\\source\\repos\\BlueByteSystemsInc\\SW_API_Tips\\");
(process 51896) exited with code 0 (0x0).
```

[Iterator] Found 124 files in 1503 ms
[Search] Found 124 files in 697 ms



SOLIDWORKS PDM API TIPS

IEdmFile5::IsLocked needs refreshing if the lock state changes.

The **IsLocked** property does not automatically update when the file's state changes in the PDM vault.

To ensure you get the current state, you must explicitly call **IEdmFile5::Refresh**. Without this step, you might work with outdated information.

```
file.Refresh();  
if (file.IsLocked)  
    // todo: do something with locked files
```

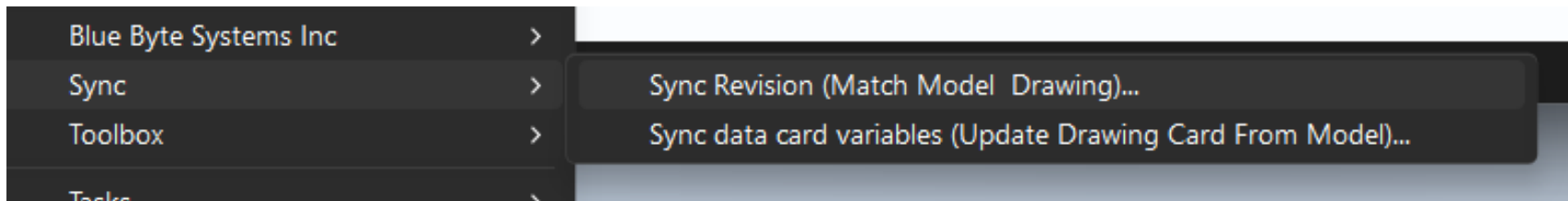
SOLIDWORKS PDM API TIPS

Creating submenus in SOLIDWORKS PDM

You can create submenus in SOLIDWORKS PDM by using the \\ separator in the menu name when calling IEdmCmdMgr5::AddCmd.

Example:

```
poCmdMgr.AddCmd(15065, "ParentMenu\\ChildMenu", 43, string.Empty,  
string.Empty, -1, 0);
```



SOLIDWORKS PDM API TIPS

Writing Add-ins with Version-Specific APIs

Always check the installed SOLIDWORKS PDM version before calling newer API methods. Use `IEdmVault5.GetVersion` to detect the major version at runtime and conditionally enable features introduced in newer releases, while providing fallback logic or a clear message for older PDM versions to prevent runtime errors.

```
void GetVersion(  
    out System.int plMajor,  
    out System.int plMinor  
)
```



SOLIDWORKS PDM API TIPS

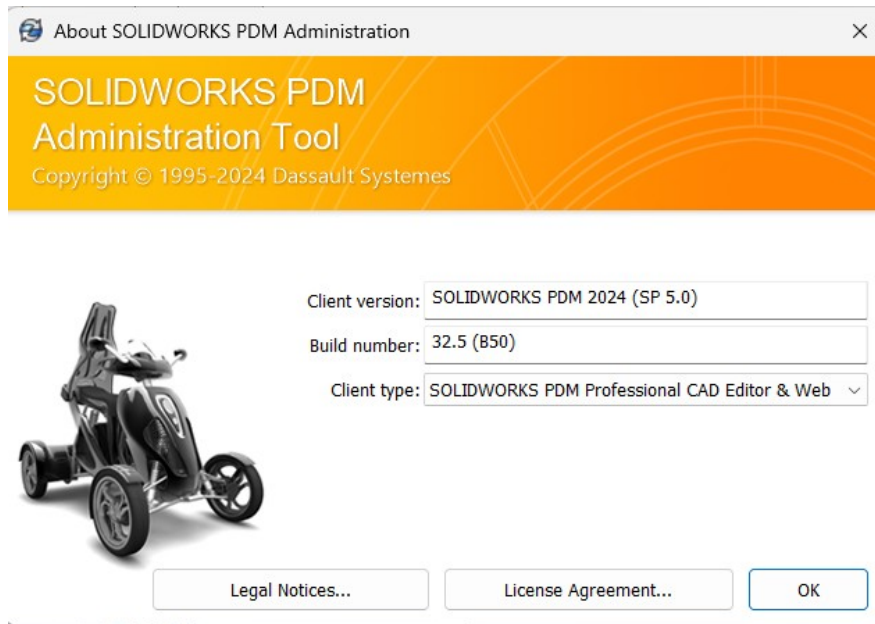
Writing Add-ins with Version-Specific APIs

In SOLIDWORKS PDM, the build number maps directly to the major release year, so you don't need a lookup table.

```
int major = 0, minor = 0;  
vault.GetVersion(ref major, ref minor);
```

```
// major = build major (e.g. 32)  
// minor = service pack (e.g. 5)
```

```
int pdmYear = major + 1992; // 32 + 1992 =  
2024  
int servicePack = minor;
```



SOLIDWORKS PDM API TIPS

Use Immutable IDs to Identify Objects in SOLIDWORKS PDM

Files and folders can be **moved** or **renamed** but IDs stay **the same**.

Always track objects by **ID** for reliable and robust code.

SOLIDWORKS PDM API TIPS

Using IEdmDictionary5 for Secure and Flexible Settings Storage in SOLIDWORKS PDM

```
var dictionary = _vault.GetDictionary("NAME", true);  
// true creates the dictionary if it does not exist  
  
// get value  
dictionary.StringGetAt("Key", out string Value);  
  
// set value  
dictionary.StringSetAt("Key", Value);
```

SOLIDWORKS PDM API TIPS

Tips for using IEdmDictionary5

- Dictionaries are stored in the vault db.
- **Do not store passwords in your a dictionary!**
- If you have to store sensitive data, make sure to at least encrypt it.

SOLIDWORKS PDM API TIPS

Optimizing Large-Scale Operations with Batch Interfaces in SOLIDWORKS PDM

Interface	Description
<code>IEdmBatchAdd2</code>	Adds multiple files or folders
<code>IEdmBatchChangeState5</code>	Transitions multiple files
<code>IEdmBatchDelete3</code>	Deletes multiple files
<code>IEdmBatchItemGeneration2</code>	Generates multiple item
<code>IEdmBatchListing4</code>	Retrieves detailed lists of files, folders, or items
<code>IEdmBatchUnlock2</code>	Checks in multiple files in
<code>IEdmBatchUpdate2</code>	Updates variables or properties for multiple files

SOLIDWORKS PDM API TIPS

Optimizing Large-Scale Operations with Batch Interfaces in SOLIDWORKS PDM

Proper sequence of calls to perform a batch process (Example: IEdmBatchUnlock2)

1. Call CreateUtility Method

```
Dim batchUnlocker As IEdmBatchUnlock2 = CType(vault.CreateUtility(EdmUtility.EdmUtil_BatchUnlock), IEdmBatchUnlock2)
```

2. Queue files in the batch object

```
batchUnlocker.AddSelection(vault, selections.ToArray())
```

3. CreateTree (Only for batch objects that needs to be aware of references)

```
batchUnlocker.CreateTree(0, EdmUnlockBuildTreeFlags.Eubtf_Nothing)
```

4. ShowDlg (This is optional)

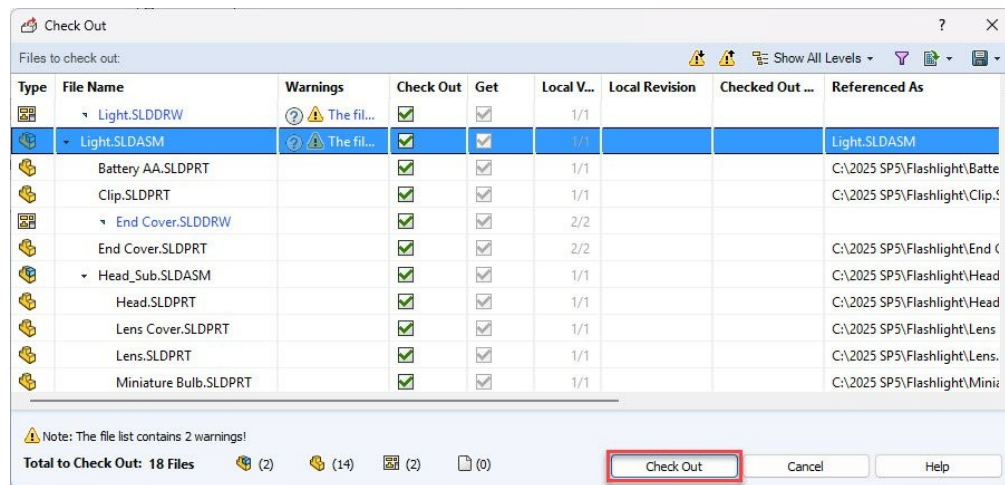
5. Call a commit method to commit the changes

```
batchUnlocker.UnlockFiles(0, Nothing)
```

SOLIDWORKS PDM API TIPS

Optimizing Large-Scale Operations with Batch Interfaces in SOLIDWORKS PDM

Some batch commands can show a dialog to user. This allows the user to make changes to the selection computed by the object. In the IEdmBatchUnlock2 (check out), you will be presented with the check out dialog:

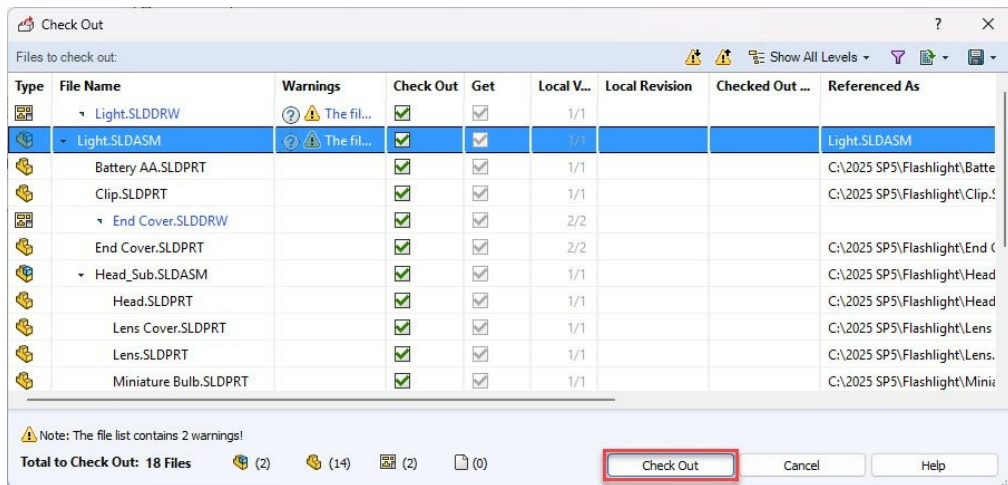


SOLIDWORKS PDM API TIPS

parent Hwnd

All API calls that optionally ask for a parent window handle need it to know where to show its user dialog. These API calls run on the **UI thread (main thread)**.

👉 **DO NOT** call these API calls from a worker or background thread in .NET

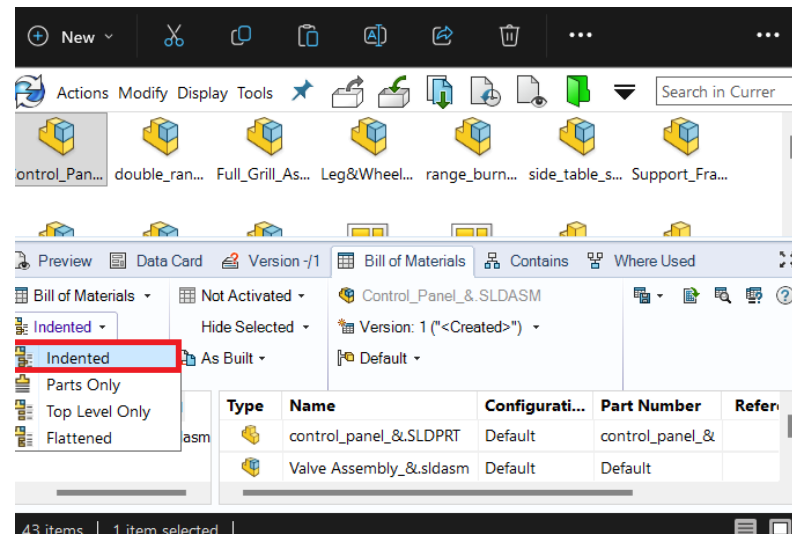


SOLIDWORKS PDM API TIPS

Extracting a "Parts Only" BOM from IEdmFile7::GetComputedBOM

When using IEdmFile7::GetComputedBOM, the method returns an indented BOM.

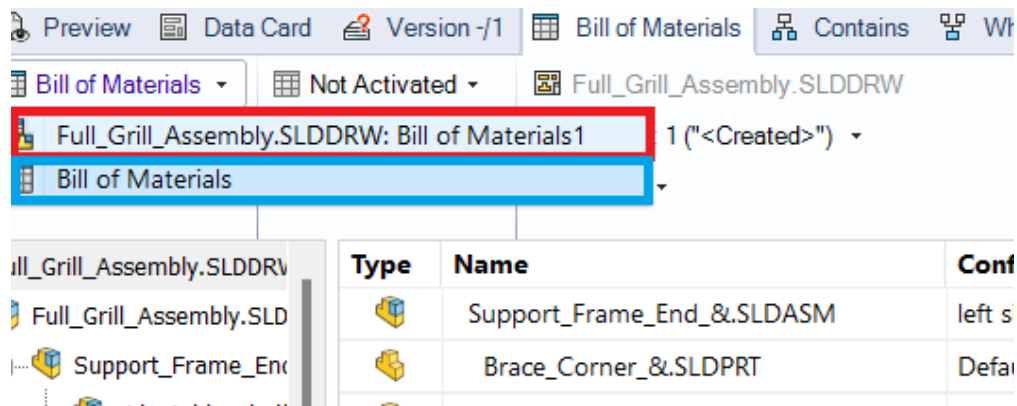
To generate a "Parts Only" BOM, you must filter out assembly nodes and retain only the leaf nodes (individual parts).



SOLIDWORKS PDM API TIPS

IEdmFile7::GetComputedBOM limitation

IEdmFile7::GetComputedBOM can only retrieve BOMs based on the BOM layout. The Bill Of Materials inside of the SOLIDWORKS files are not supported



From SOLIDWORKS file
Computed from SOLIDWORKS PDM

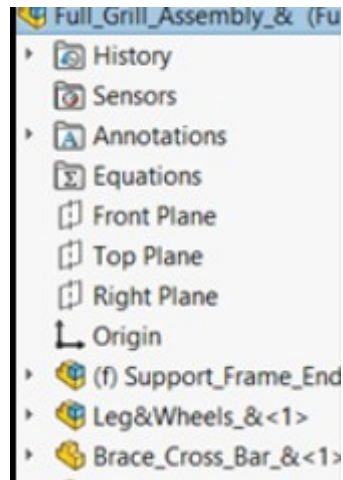
SOLIDWORKS PDM API TIPS

Considerations for using RenameEx Method (IEdmFile6)

- BEFORE RENAMING, EVERYONE MUST CHECK THEIR FILES INTO THE VAULT.
- Make sure everyone's local cache is cleared before starting the rename process. This avoids previous cache showing as private files.
- SOLIDWORKS component tree is not automatically updated (with new names) for assemblies. A rebuild is required.

RenameEx Method (IEdmFile6)

Changes the name of this file.



SOLIDWORKS PDM API TIPS

You can actually cancel all hooks that start with Pre

There are about 21 hooks that start with Pre that are cancellable by the API.

- EdmCardControlType
- EdmCardFlag
- EdmCardType
- EdmCardViewFlag
- EdmCardViewUpdateType
- EdmChangeStateFileListFlag
- EdmClientType
- EdmCmdNodeProp
- **EdmCmdType**
- EdmColType
- EdmCopyFlag
- EdmCopyType
- EdmCreateListExFlags
- EdmCreateReferenceFlags
- EdmCreateVaultFlag
- EdmCreateVaultViewFlag
- EdmDataPropertyType
- EdmDataType
- EdmDefaultValueType

Member	
EdmCmd_ActivateAPITab	57 =
EdmCmd_CardButton	37 =
EdmCmd_CardInput	38 =
EdmCmd_CardListSrc	39 =
EdmCmd_DeSelectItem	56 =
EdmCmd_InstallAddIn	23 =
EdmCmd_Menu	1 =
EdmCmd_PostAdd	12 =
EdmCmd_PostAddFolder	28 =
EdmCmd_PostCopy	20 =
EdmCmd_PostCopyFolder	36 =
EdmCmd_PostDelete	14 =
EdmCmd_PostDeleteFolder	30 =
EdmCmd_PostGet	26 =
EdmCmd_PostLabel	47 =



SOLIDWORKS PDM API TIPS

Cancelling transitions in PDM with the API

If your add-in listens to the PreState hook (when files are about to change state)

```
// listen to state changes  
poCmdMgr.AddHook(EdmCmdType.EdmCmd_PreState);
```

You can cancel the state change during the OnCmd execution by specifying
poCmd.mbCancel = (short)1;

```
public void OnCmd(ref EdmC  
{  
    poCmd.mbCancel = 1;
```

SOLIDWORKS PDM API TIPS

Cancelling transitions in PDM with the API (C#)

```
public void OnCmd(ref EdmCmd poCmd, r
{
    // this does not work in c#
    poCmd.mbCancel = 1;
    // this works
    poCmd.mbCancel = (short)1;
```

In C#, you need to explicit convert 1 to a short so that it can be marshalled (converted) correctly to the value PDM understands.

PROGRAMMING BEST PRACTICES

API documentation remarks section

The online documentation should be the first place to check for API calls.

Beyond the syntax, always read the **Remarks** section. It often explains:

- Important behavior,
- limitations,
- **Call sequence**,
- and edge cases.

Remarks

This interface represents a SOLIDWORKS PDM Professional file vault. It is the

[Standalone programs](#) using this API should first declare and create an instance

The type library includes a creatable class called EdmVault5, which implements

You can create an instance of the EdmVault5 class with the **New** keyword, ; created, most of the other interface objects within this API are returned either

To access this interface, stand-alone programs can:

Obtain an instance through one of the accessors.

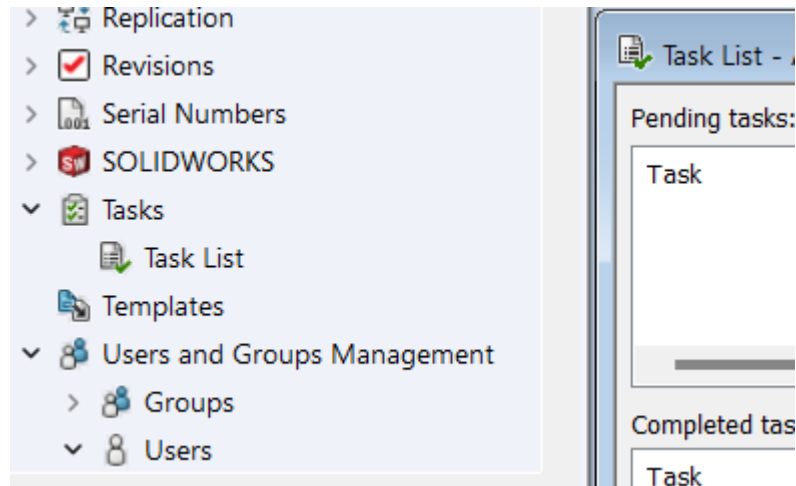
Create an instance:

```
Dim vault as EdmVault5  
vault = new EdmVault5
```

SOLIDWORKS PDM API TIPS

Running SOLIDWORKS PDM Tasks from Standalone Applications

Run SOLIDWORKS PDM tasks directly from standalone applications using `IEdmTaskMgr.RunTask`, enabling external tools to trigger task add-ins without creating a full PDM add-in.



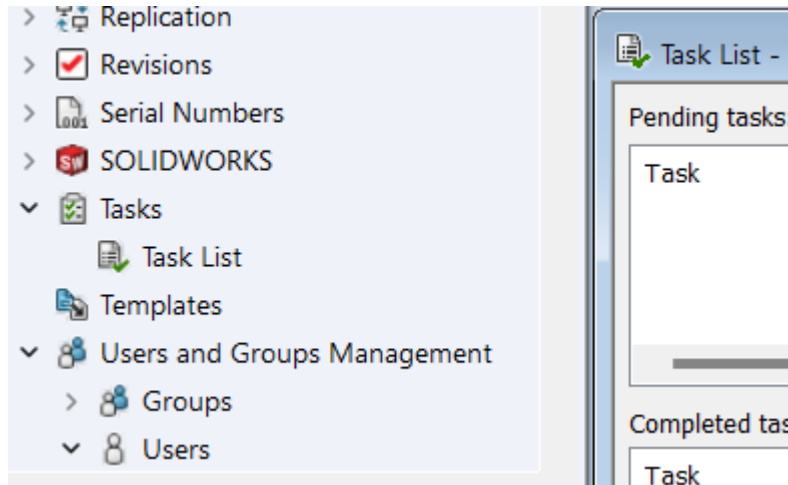
SOLIDWORKS PDM API TIPS

Understanding How SOLIDWORKS PDM Task Add-Ins Are Launched and Run

Tasks in SOLIDWORKS PDM do not run inside the PDM client (***File Explorer***).

They execute in separate processes:

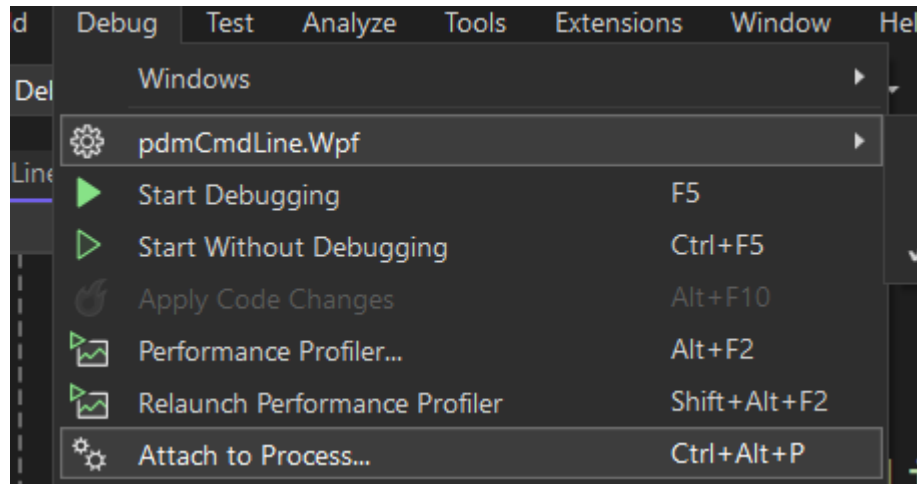
- TaskLauncher.exe (TaskLaunch)
 - TaskExecutor.exe (TaskRun)
- which improves isolation but makes debugging more difficult.



SOLIDWORKS PDM API TIPS

Debugging SOLIDWORKS PDM Task Add-Ins When Launched and Run

- Add a message box call printing the process ID
(System.Diagnostics.Process.GetRunningProcess().Id) in code
- Compile DLLs
- Add add-in (***Not debug add-ins***)
- MessageBox shows up with the ID of the process that hosting the add-in
- In Visual Studio, Click *Debug* > *Attach to Process*

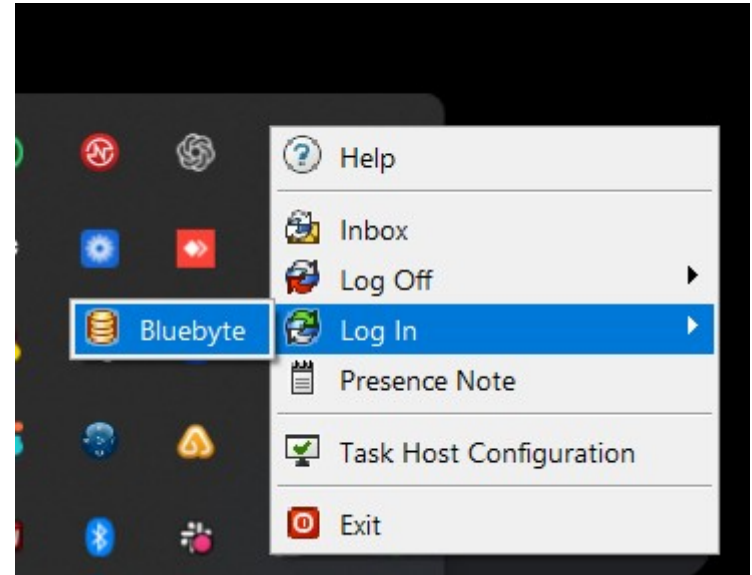


Things you can't do with the API

Logout

While, you can log from a vault from the icon tray, there is no official API call to simulate this behavior.

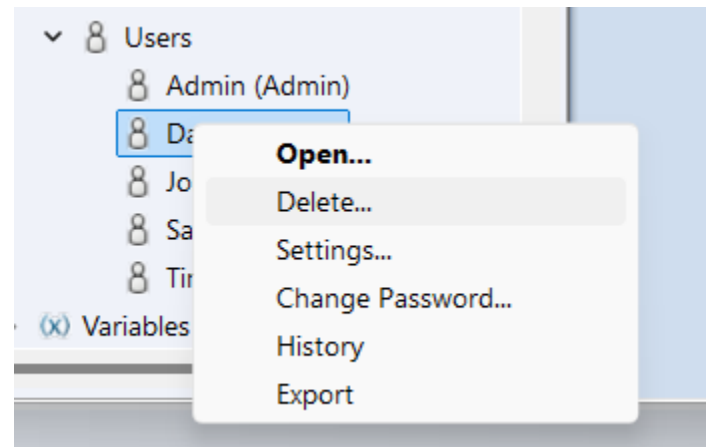
This would be extremely handy for standalone applications.



Things you can't do with the API

Delete user

There are many API to list and edit the settings of user but none via the API.



Things you can't do with the API

Edit and delete column in the BOM layout editor

There are a number of API calls that would list the BOM layouts but none that allow editing or delete the columns from their BOM layout definition.

Bill of materials name: Bill of Materials Type:

☐ Include derived part references
☐ Include cut list references

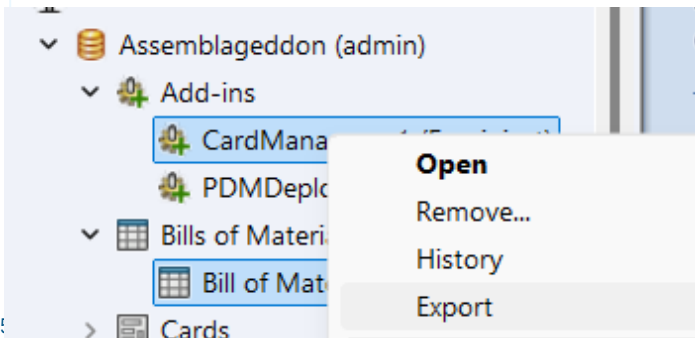
Preview:

Name	Configuration	Part Number	Reference C...	Material
------	---------------	-------------	----------------	----------

Columns:

Variable	Column Name	Width	Alignment
<Name>	Name	100	Left
<Configuratio...	Configuration	100	Left
<Part Number>	Part Number	100	Left
<Reference C...	Reference C...	100	Right
Material	Material	100	Left

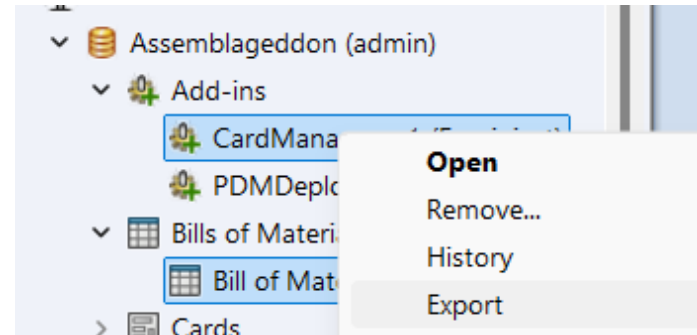
New Column Delete Column



Things you can't do with the API

Export vault settings to CEX files

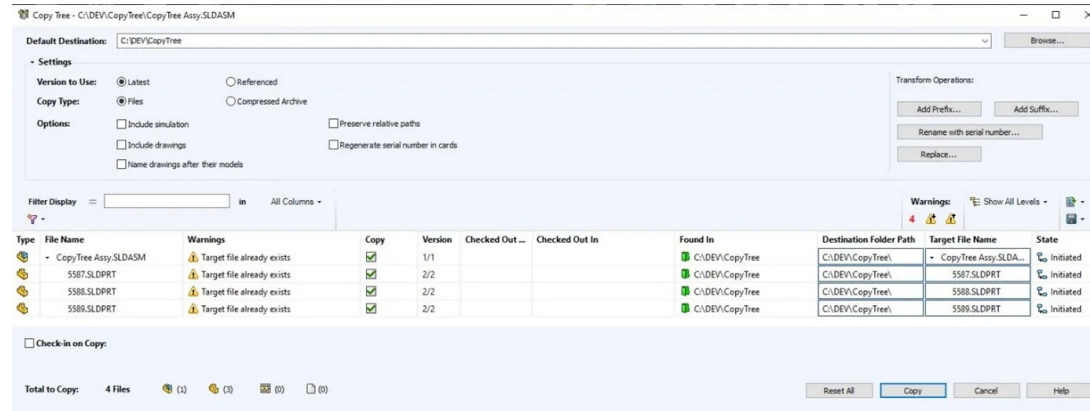
There is an API call to import a CEX file into the vault but none to export vault settings.



Things you can't do with the API

Copy tree is limited

PDM API exposes a call to perform copy tree but it only works with **assemblies** (no parts or drawings)



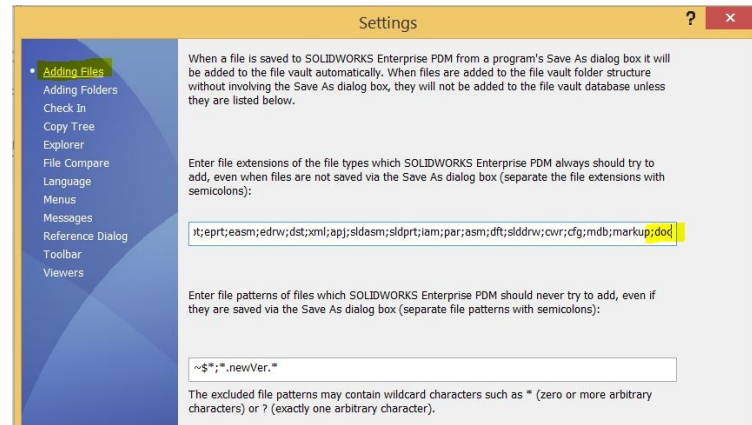
SOLIDWORKS PDM API TIPS

Avoid race conditions when saving files directly to the vault

When a program saves a file into a PDM-managed folder, PDM may not immediately finish adding the file to the vault. If the program calls `IEdmVault5::GetFileFromPath` before PDM completes this process, the call can fail because the file is not yet recognized by the vault.

This issue is timing-dependent and occurs when the program runs faster than PDM's background file-add operation.

👉 With auto-added extensions, It's better to save the file to a temp folder and then add it to PDM using `IEdmFolder5::AddFile`



VISUAL STUDIO ENVIRONMENT

UAC User profile

- Make sure to run Visual Studio as the local user.
- Add-in DLLs are registered by the Administration tool. Administration is launched normally as the local user.
- **Avoid running VS as Windows Administrator. This will cause a registration bug during the debugging process. Your debug program (explorer.exe or notepad.exe) will attempt to load the add-in in memory using the user's UAC level.**

AFTER YOU UPGRADE PDM

Add-ins should not be affected by upgrades **BUT**

- It is highly recommended to **recompile** your add-ins and **test** them with the latest version PDM prior to the upgrade.
- Update the **latest** PDM Primary Interop Assemblies (PIAs)
- Create a snapshot of the existing add-ins as a CEX for your backups

Link To Presentation

Download the presentation slides from

bluebytesystems.com/3dx



YOUR VOICE MATTERS!

PLEASE FILL OUT THIS
SESSION'S
SURVEY IN THE MOBILE APP

The logo for 3DEXPERIENCE WORLD is positioned on the right side of the slide. It features the word "3DEXPERIENCE" in a smaller, white, sans-serif font, with a registered trademark symbol (®) to its upper right. Below it, the word "WORLD" is written in a much larger, bold, white, sans-serif font. The background of the logo area is a dark blue circle containing a network of white dots connected by thin white lines, resembling a molecular or digital structure. In the top right corner of the slide, there is a circular graphic with concentric white lines and a red and blue color scheme.

3DEXPERIENCE[®]
WORLD

3DEXPERIENCE[®] WORLD

